

1. cv2.blur() — Averaging Blur

This is the simplest blur method. It calculates the average of all pixels in the kernel area and replaces the central pixel with that average. It is a normalized box filter (the kernel is divided by its area, so the sum of all weights equals 1).

cv2.blur(src, ksize, dst=None, anchor=(-1, -1), borderType=cv2.BORDER_DEFAULT)

Parameters & Options

Parameter	Type	Required?	Default	Description	Available Options
src	InputArray	Yes	—	Input image. Can be single-channel or multi-channel.	Any valid image array
ksize	Size	Yes	—	Blurring kernel size.	Positive odd integer pair (width, height), e.g., (5, 5), (31, 31)
dst	Output Array	No	None	Output image of the same size and type as src.	—
anchor	Point	No	(-1, -1)	Anchor point of the kernel. Default (-1, -1) means the anchor is at the kernel center.	Any (x, y) point within the kernel bounds
borderType	int	No	cv2.BORDER_DEFAULT	Pixel extrapolation method for border pixels.	See Border Types section below

Exam point: ksize must be positive and odd. If you pass an even number, OpenCV will either error or round it to the nearest odd value.

2. cv2.boxFilter() — Box Filter

This is similar to `cv2.blur`, but it gives you control over whether the kernel is normalized. When `normalize=True`, it behaves exactly like `cv2.blur`. When `normalize=False`, the kernel is not divided by its area, so the output values are the sum of the neighboring pixels (not the average).

`cv2.boxFilter(src, ddepth, ksize, dst=None, anchor=(-1, -1), normalize=True, borderType=cv2.BORDER_DEFAULT)`

Parameters & Options

Parameter	Type	Required?	Default	Description	Available Options
src	InputArray	Yes	—	Input image.	Any valid image array
ddepth	int	Yes	—	Desired depth of the destination image.	-1 (same as source), cv2.CV_8U, cv2.CV_16S, cv2.CV_16U, cv2.CV_32F, cv2.CV_64F
ksize	Size	Yes	—	Blurring kernel size.	Positive odd integer pair (width, height)
dst	Output Array	No	None	Output image.	—
anchor	Point	No	(-1, -1)	Anchor point.	Any (x, y) within kernel
normalize	bool	No	True	Whether the kernel is normalized by its area.	True (divide by area), False (no division)
borderType	int	No	cv2.BORDER_DEFAULT	Border extrapolation method.	See Border Types below

Exam point: `normalize=False` is useful for computing integral characteristics like covariance matrices of image derivatives (used in dense optical flow).

3. cv2.GaussianBlur() — Gaussian Blur

This is the most widely used blur. It applies a Gaussian kernel, meaning the central pixel gets the highest weight, and weights decrease as distance from the center increases. It is highly effective at reducing Gaussian noise while preserving overall structure.

cv2.GaussianBlur(src, ksize, sigmaX, dst=None, sigmaY=0, borderType=cv2.BORDER_DEFAULT)

Parameters & Options

Parameter	Type	Required?	Default	Description	Available Options
src	InputArray	Yes	—	Input image. Depth should be CV_8U, CV_16U, CV_16S, CV_32F, or CV_64F.	Any valid image array
ksize	Size	Yes	—	Gaussian kernel size.	Positive odd integer pair (width, height). Can be different for width and height.
sigmaX	double	Yes	—	Standard deviation in the X direction.	Any positive real number. If 0, it is computed from ksize.width.
dst	Output Array	No	None	Output image.	—
sigmaY	double	No	0	Standard deviation in the Y direction.	Any positive real number. If 0, it is set equal to sigmaX. If both are 0, they are

					computed from ksize.
borderType	int	No	cv2.BORDER_DEFAULT	Border extrapolation method.	See Border Types below

Exam point: If ksize is (0, 0), OpenCV computes the kernel size from sigmaX and sigmaY. It is recommended to specify all of ksize, sigmaX, and sigmaY explicitly.

4. cv2.medianBlur() — Median Blur

This filter replaces each pixel with the median of the pixel values in its neighborhood. It is particularly effective at removing salt-and-pepper noise (random black and white pixels) while preserving edges better than linear filters.

cv2.medianBlur(src, ksize, dst=None)

Parameters & Options

Parameter	Type	Required?	Default	Description	Available Options
src	InputArray	Yes	—	Input image. Can be 1, 3, or 4-channel. When ksize > 5, the image depth must be CV_8U.	Any valid image array
ksize	int	Yes	—	Kernel size. Must be an odd integer greater than 1.	3, 5, 7, 9, etc. (odd numbers only)
dst	OutputArray	No	None	Output image of the same type and depth as src.	—

Exam point: medianBlur uses BORDER_REPLICATE internally for border pixels, so you cannot change the border type.

5. cv2.bilateralFilter() — Bilateral Filter

This is an advanced filter that considers both the spatial distance and the color/intensity difference of neighboring pixels. This means it smooths flat areas but preserves edges, making it ideal for noise reduction where edge preservation is critical.

cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace, dst=None, borderType=cv2.BORDER_DEFAULT)

Parameters & Options

Parameter	Type	Required?	Default	Description	Available Options
src	InputArray	Yes	—	Source 8-bit or floating-point, 1-channel or 3-channel image.	Any valid image array
d	int	Yes	—	Diameter of each pixel neighborhood used during filtering.	Any positive integer. If non-positive, it is computed from sigmaSpace. Recommended: 5 for real-time, 9 for heavy noise filtering.
sigmaColor	double	Yes	—	Filter sigma in the color space. Larger values mean farther colors are mixed together.	Any positive real number. Values < 10 = little effect. Values > 150 = strong "cartoonish" effect.
sigmaSpace	double	Yes	—	Filter sigma in the coordinate space. Larger values mean farther pixels influence each other (if colors are close enough).	Any positive real number. When d > 0, it specifies neighborhood size regardless of sigmaSpace.

dst	Output Array	No	None	Output image.	—
borderType	int	No	cv2.BORDER_DEFAULT	Border extrapolation method.	See Border Types below

Exam point: Bilateral filter is very slow compared to other filters. It does not work in-place. For simplicity, you can set sigmaColor and sigmaSpace to the same value.

6. cv2.filter2D() — Custom Convolution

This is the most general function. You can create your own custom kernel (a 2D matrix) and apply it to the image. All the blurring functions above are essentially special cases of this function with specific kernels.

cv2.filter2D(src, ddepth, kernel, dst=None, anchor=(-1, -1), delta=0, borderType=cv2.BORDER_DEFAULT)

Parameters & Options

Parameter	Type	Required?	Default	Description	Available Options
src	InputArray	Yes	—	Input image.	Any valid image array
ddepth	int	Yes	—	Desired depth of the destination image.	-1 (same as source), cv2.CV_8U, cv2.CV_16S, cv2.CV_16U, cv2.CV_32F, cv2.CV_64F
kernel	InputArray	Yes	—	Convolution kernel (a single-channel floating-point matrix).	Any 2D NumPy array of type np.float32. E.g., np.ones((5,5), np.float32)/25 for averaging.

dst	Output Array	No	None	Output image of the same size and number of channels as src.	—
anchor	Point	No	(-1, -1)	Anchor of the kernel indicating the relative position of the filtered point.	Any (x, y) within the kernel. Default = kernel center.
delta	double	No	0	Optional value added to the filtered pixels before storing them in dst.	Any real number. Useful for adjusting brightness.
borderType	int	No	cv2.BORDER_DEFAULT	Border extrapolation method.	See Border Types below

Exam point: filter2D can apply different kernels to different channels by splitting the image into separate color planes first.

Common Border Types (for borderType Parameter)

The following options are available for the borderType parameter in all functions above. They define how pixels outside the image boundary are extrapolated.

Constant	Value	Behavior	Example (for abcdefgh)
cv2.BORDER_CONSTANT	0	Adds a constant colored border (specified separately).	iiii abcdefgh iiii
cv2.BORDER_REPLICATE	1	Replicates the last element.	aaaaa abcdefgh hhhh
cv2.BORDER_REFLECT	2	Mirrors the border pixels, including the edge.	fedcba abcdefgh hgfedcb

cv2.BORDER_WRAP	3	Wraps around the image.	cdefgh abcdefgh abcdefgh
cv2.BORDER_REFLECT_101	4	Mirrors the border pixels, excluding the edge pixel.	gfedcb abcdefgh gfedcba
cv2.BORDER_DEFAULT	4	Same as BORDER_REFLECT_101.	gfedcb abcdefgh gfedcba
cv2.BORDER_TRANSPARENT	5	Pixels outside are not modified.	—
cv2.BORDER_ISOLATED	16	Do not look outside the image ROI.	—

Note: BORDER_WRAP is not supported by GaussianBlur and boxFilter.
Exam point: BORDER_DEFAULT is an alias for BORDER_REFLECT_101. This is the most commonly used border type.

Quick Comparison Table

Function	Kernel Type	Preserves Edges?	Best For	Key Parameter
cv2.blur()	Normalized box	No	Simple smoothing	ksize
cv2.boxFilter()	Box (optional normalization)	No	Customizable averaging	normalize
cv2.GaussianBlur()	Gaussian	No (but better)	General noise reduction	sigmaX, sigmaY
cv2.medianBlur()	Median	Yes	Salt-and-pepper noise	ksize (odd int)
cv2.bilateralFilter()	Bilateral	Yes	Noise reduction with edge preservation	sigmaColor, sigmaSpace

cv2.filter2D()	Custom	Depends on kernel	Any custom filtering	kernel
----------------	--------	----------------------	-------------------------	--------
